

Conference Room Booking API — коротка документація

Кандидат: Роман Мельничук

Репозиторій: <https://github.com/RomanMelnichuk/ConferenceRoomBooking>

Інструкція із запуску, повний опис API та рішень — у README репозиторію.

1. Бізнес-завдання

Компанія здає конференц-зали в оренду бізнесу. API обслуговує дві групи користувачів:

Хто	Завдання	Як вирішено
Клієнти	Знайти зал потрібної місткості, вільний у потрібний час	Пошук вільних залів за датою, часом і місткістю
Клієнти	Забронювати зал з послугами й одразу знати ціну	Бронювання з автоматичним розрахунком вартості
Компанія	Керувати залами та їхніми послугами	Додавання, редагування, видалення залів; каталог послуг з назвою та ціною
Компанія	Розуміти, що приносить гроші, а що простоє	Звіти по залах і по послугах

Ціноутворення. Вартість залежить від часу: ранок 06:00–09:00 –10%, стандарт 09:00–18:00, пік 12:00–14:00 +15%, вечір 18:00–23:00 –20%. Ціна рахується **погодинно**, бо одне бронювання може зачіпати кілька тарифних зон. Послуги оплачуються один раз за бронювання. Приклад: зал А (2000 грн/год) з 11:00 до 13:00 з проектором = 2000 + 2300 (пік) + 500 = 4800 грн.

2. Звіти для бізнесу

Звіт	Що показує	Яке рішення допомагає ухвалити
По залах	Кількість бронювань, заброньовані години, виручка, завантаженість у %	Які зали приносять гроші, а які простоюють: знижки або інше використання
По послугах	Скільки разів замовили послугу і скільки вона принесла	У які послуги інвестувати, а які прибрати
По тарифних зонах	Скільки заброньованих годин припадає на ранок, стандарт, пік і вечір	Чи справді знижки приваблюють клієнтів і наскільки завантажений пік: основа для зміни коефіцієнтів

3. Ключові технічні рішення

- **Архітектура:** контролер → сервіс → DbContext; бізнес-логіка в сервісах за інтерфейсами, впроваджених через DI.
- **Ціноутворення в одному класі:** тарифи — таблиця зон у PriceCalculator, логіка покрита юніт-тестами (xUnit).
- **Ціна зберігається в бронюванні:** зміна тарифу не перераховує вже зроблені бронювання.
- **decimal для грошей:** double зберігає числа у двійковому вигляді й накопичує похибку.
- **Перевірка вільного часу:** інтервали перетинаються, якщо $start1 < end2$ і $start2 < end1$. Фільтрація виконується в SQL, запит прискорює складений індекс (RoomId, StartTime, EndTime).
- **Захист від подвійного бронювання:** перевірка й вставка в одній транзакції з рівнем ізоляції Serializable.
- **Єдиний формат помилок:** глобальний обробник повертає ProblemDetails (RFC 7807) зі зрозумілим поясненням.

4. Безпека, відмовостійкість, масштабованість

- **Ключ API** для операцій компанії (керування залами й послугами, звіти); порівняння ключа за сталий час; у production ключ задається змінною середовища, а не зберігається в репозиторії.
- **Валідація вхідних даних**, обмеження частоти запитів (100 за хвилину з однієї IP), приховування внутрішніх деталей помилок, параметризовані SQL-запити, обмеження періоду звітів до 366 днів.

- **Автоматичні повтори** при короткочасних збоях бази, включно з транзакцією бронювання; **health check** /health для моніторингу.
- **Масштабованість:** API без стану в пам'яті (можна запускати кілька копій), async, бізнес-правила в окремих класах, DTO відділяють API від структури бази.

5. Припущення там, де ТЗ неоднозначне

- Пікові години замінюють стандартний тариф, коефіцієнти не сумуються.
- Бронювання з 23:00 до 06:00 заборонене (години не описані в ТЗ).
- Бронювання лише цілими годинами, в межах одного дня і не в минулому (тому приклад із ТЗ з датою 01.09.2024 поверне помилку 400).
- Послуги — спільний каталог; зал має набір доступних послуг; бронювати можна лише доступні в залі.
- Зал із бронюваннями видалити не можна: інакше зникла б історія і звіти стали б неточними.
- Облікових записів клієнтів у ТЗ немає, тому бронювання анонімні.

6. Можливі наступні кроки

Облікові записи клієнтів і JWT з ролями, скасування бронювання, збереження ціни послуг у бронюванні, налаштування тарифів без перезбірки, спільне сховище лімітів запитів (Redis) для кількох копій API, інтеграційні тести.